🧠 INPUT

```
(batch, 3, 128, 128)
```

🔥 BLOCK 1

1️⃣ Conv Layer

```
nn.Conv2d(in_channels=3, out_channels=32, kernel_size=3, stride=1, padding='same')
```

What parameters mean:

- `in_channels=3` → RGB input
- `out_channels=32` → 32 filters
- `kernel_size=3` → 3×3 filter
- `stride=1` → move 1 pixel
- `padding='same'` → keep size same

Shape:

```
(3, 128, 128) → (32, 128, 128)
```

2️⃣ ReLU

```
nn.ReLU(inplace=False)
```

- No parameters affecting shape

Shape:

```
(32, 128, 128) → (32, 128, 128)
```

3️⃣ BatchNorm

```
nn.BatchNorm2d(num_features=32)
```

- `num_features=32` → normalize each channel

Shape:

```
(32, 128, 128) → (32, 128, 128)
```

4️⃣ MaxPool

```
nn.MaxPool2d(kernel_size=2, stride=2)
```

- `kernel_size=2` → 2×2 window
- `stride=2` → jump 2 pixels

Shape:

```
(32, 128, 128) → (32, 64, 64)
```

🔥 BLOCK 2

5️⃣ Conv

```
nn.Conv2d(32, 64, kernel_size=3, stride=1, padding='same')
```

- Input channels = 32
- Output = 64

Shape:

```
(32, 64, 64) → (64, 64, 64)
```

6️⃣ ReLU + BatchNorm

```
(64, 64, 64) → (64, 64, 64)
```

7️⃣ MaxPool

```
nn.MaxPool2d(2)
```

Shape:

```
(64, 64, 64) → (64, 32, 32)
```

🔥 BLOCK 3

8️⃣ Conv

```
nn.Conv2d(64, 128, kernel_size=3, stride=1, padding='same')
```

Shape:

```
(64, 32, 32) → (128, 32, 32)
```

9️⃣ ReLU + BatchNorm

```
(128, 32, 32) → (128, 32, 32)
```

🔟 MaxPool

```
nn.MaxPool2d(2)
```

Shape:

```
(128, 32, 32) → (128, 16, 16)
```

🎯 BEFORE FLATTEN

```
(128, 16, 16)
```

🔥 FLATTEN

```
nn.Flatten(start_dim=1)
```

Meaning:

- Convert 3D → 1D

Calculation:

```
128 × 16 × 16 = 32768
```

Shape:

```
(128, 16, 16) → (32768)
```

With batch:

```
(batch, 128, 16, 16) → (batch, 32768)
```

🧠 FINAL FLOW (VERY IMPORTANT)

```
Input       → (3, 128, 128)
Conv1       → (32, 128, 128)
Pool        → (32, 64, 64)
Conv2       → (64, 64, 64)
Pool        → (64, 32, 32)
Conv3       → (128, 32, 32)
Pool        → (128, 16, 16)
Flatten     → (32768)
```

🔥 KEY INSIGHT

Why this pattern?

Operation

Effect

Conv

↑ channels (more features)

Pool

↓ spatial size

Flatten

convert to vector